

DNA Sequencing analysis on Artemis

Mapping and Variant Calling

Tracy Chew

Senior Research Bioinformatics Technical Officer

Rosemarie Sadsad

Informatics Services Lead

Hayim Dar

Informatics Technical Officer

Nathaniel Butterworth

Senior Research Informatics Technical Officer

Sydney Informatics Hub

sih.info@sydney.edu.au



By the end of this course, you will...

- Be able to run a bioinformatics pipeline on Artemis
- Gain confidence with editing and submitting PBS scripts as jobs
- Understand concepts of analysis methodologies and considerations that should be taken when designing your own pipeline
- Interpret common file formats and ways to interrogate them
- Using job arrays in PBS to process multiple jobs in parallel
- Using the interactive node on Artemis

Prerequisites: Intro to Artemis or some command line knowledge

Some tips

- I have included full paths but if you are confident with the command line feel free to use some shortcuts (e.g. “..”)
- When typing a path or filename on the command line, use tab to autocomplete, or double tab to ls
- When you see <word>, replace everything including the brackets, with whatever is relevant to you
- The command line is by default **case sensitive** (and typo sensitive)!
- It is also sensitive to **spaces and newlines**

Introduction

Today we will call variants in the gene *BADH2* to determine if our rice (*Oryza sativa Indica*) has any fragrance alleles
(simulated data)

[Molecular Breeding](#)

November 2015, 35:217 | [Cite as](#)

Discovery of a novel fragrant allele and development of functional markers for fragrance in rice

Authors

[Authors and affiliations](#)

Qiang He, Yong-Jin Park 

Article

First Online: 11 November 2015

486

Downloads

4

Citations

Abstract

Fragrance in rice (*Oryza sativa* L.) results from the loss of function of the betaine aldehyde dehydrogenase (*Badh2*) gene on chromosome 8. An 8-bp deletion in exon 7 of *Badh2* was reported to be the main allele functionally associated with fragrance. The discovery of new functional alleles will provide additional genetic resources to improve fragrant rice. In this



Course outline

Part A: Getting started

- Logging on
- FASTQ and FASTA files

Part B: Quality checking

- FastQC
- Introduction to PBS job arrays
- Introduction to interactive node on Artemis

Part C: Preparing the reference genome

- Indexing the reference genome

Part D: Reference mapping

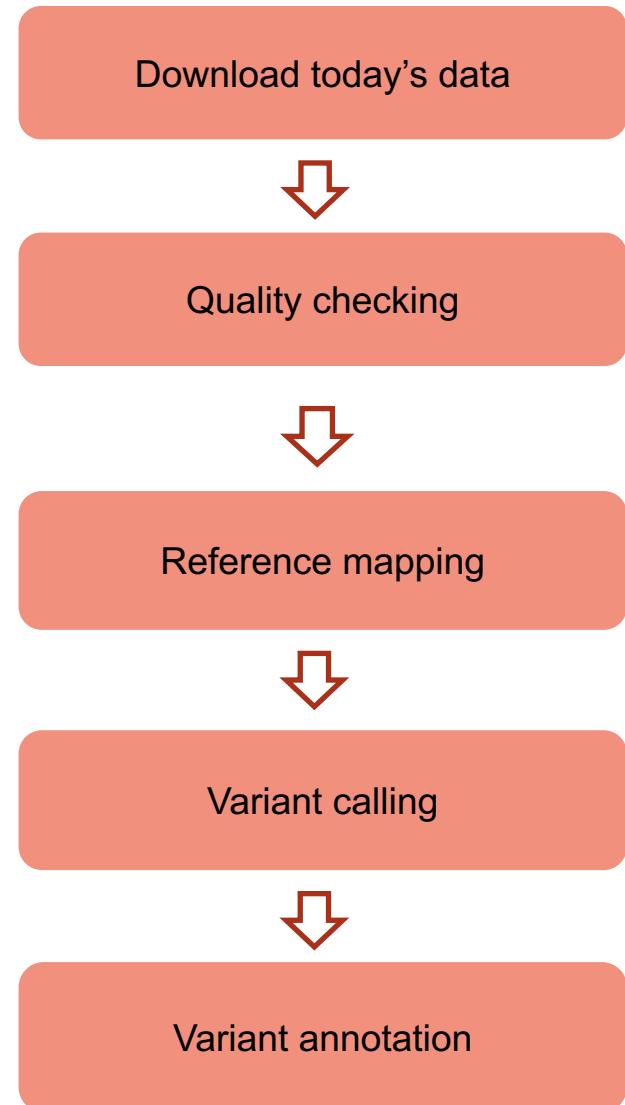
- Mapping workflow: BWA-mem, mark duplicates, realign around indels, BQSR
- Alignment stats and visualization
- BAM/SAM files

Part E: Variant calling

- Variant calling with GATK Haplotype Caller

Part F: Variant annotation

- Annotation with VEP



Training unikey

Today we will assign training unikeys for you to use in this course.

The training unikey is:

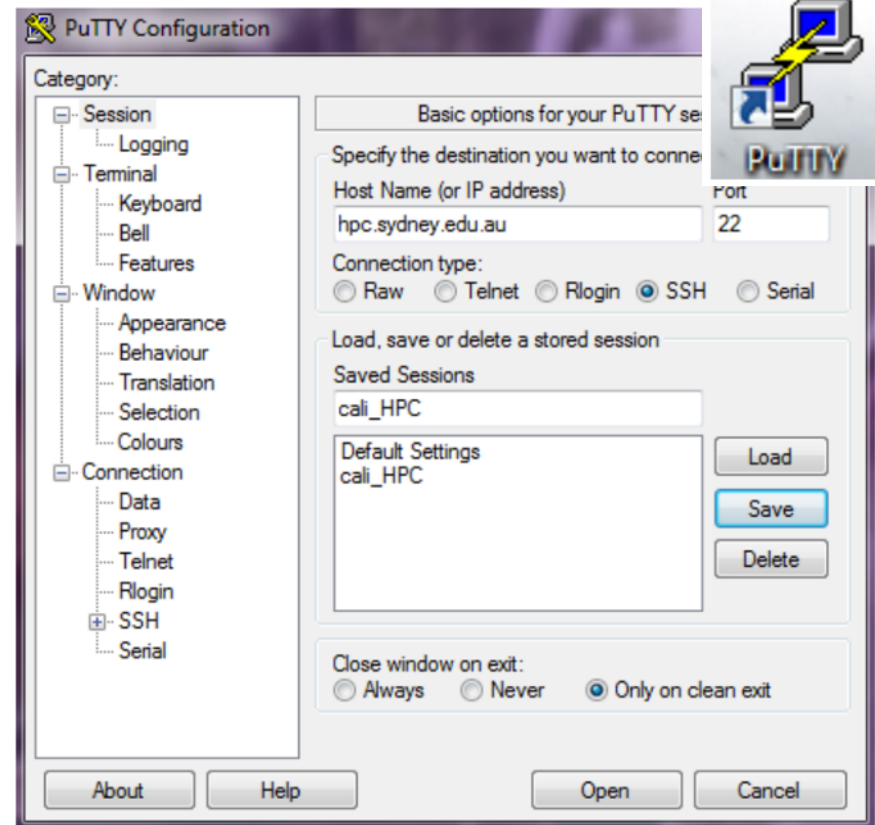
ict_hpctrainN

(N = 1–40, we will assign you a number)

Terminal client

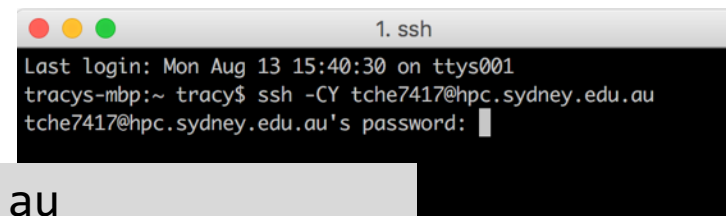
Windows users

- Go to: <http://www.putty.org/>
- Download and run 'putty.exe'
- In the configuration window, enter the following:
 - Under 'Host Name': hpc.sydney.edu.au
 - Leave 'Port' as 22
 - Open SSH category:
 - Enable compression
 - X11: Tick enable X11 forwarding
 - Click 'Open'
 - At 'login as' enter your training unikey
 - Enter the training unikey password



Mac users

- Go > Utilities > Terminal
- XQuartz or iTerm2 to ssh with X11 forwarding
- Type command below, followed by the password



```
ssh -CY ict_hpctrainN@hpc.sydney.edu.au
```

Part A: Getting the data

Please create a directory to work in (or cd into an existing one):

```
cd /project/Training
```

```
mkdir <unikey>
```

```
cd <unikey>
```

To download the data for this workshop (please type):

```
wget -O DNA_workshop.tar.gz <download_URL>
```

Replace <download_URL> with (you can copy this part):

<https://cloudstor.aarnet.edu.au/plus/s/efreY0K44DoAY4w/download>

Unzip and unpack the tar file:

```
tar -xzf DNA_workshop.tar.gz
```

Remove the tar file

```
rm -xzf DNA_workshop.tar.gz
```

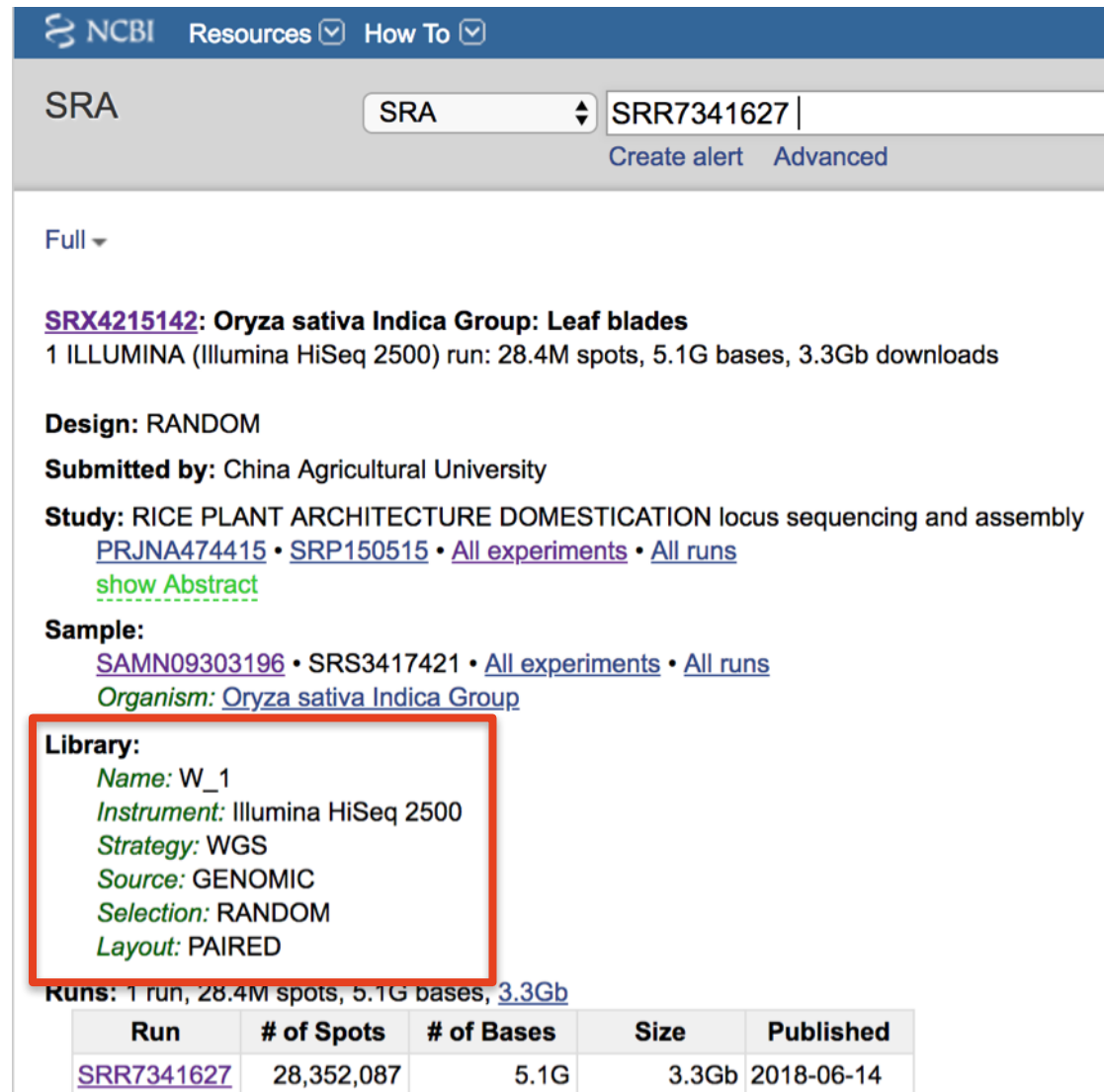
Part A: Getting to know your data

Oryza sativa Indica

- Diploid
- ~500 Mbp genome
- $n = 12$

SRR7341627

To ensure your scripts run to completion during the training course, this data has been sub-sampled to ~84 Mb on chromosome 8 (*BADH2* region) and partially modified



NCBI Resources How To

SRA SRA SRR7341627 Create alert Advanced

Full

SRX4215142: Oryza sativa Indica Group: Leaf blades
1 ILLUMINA (Illumina HiSeq 2500) run: 28.4M spots, 5.1G bases, 3.3Gb downloads

Design: RANDOM

Submitted by: China Agricultural University

Study: RICE PLANT ARCHITECTURE DOMESTICATION locus sequencing and assembly
[PRJNA474415](#) • [SRP150515](#) • [All experiments](#) • [All runs](#)
[show Abstract](#)

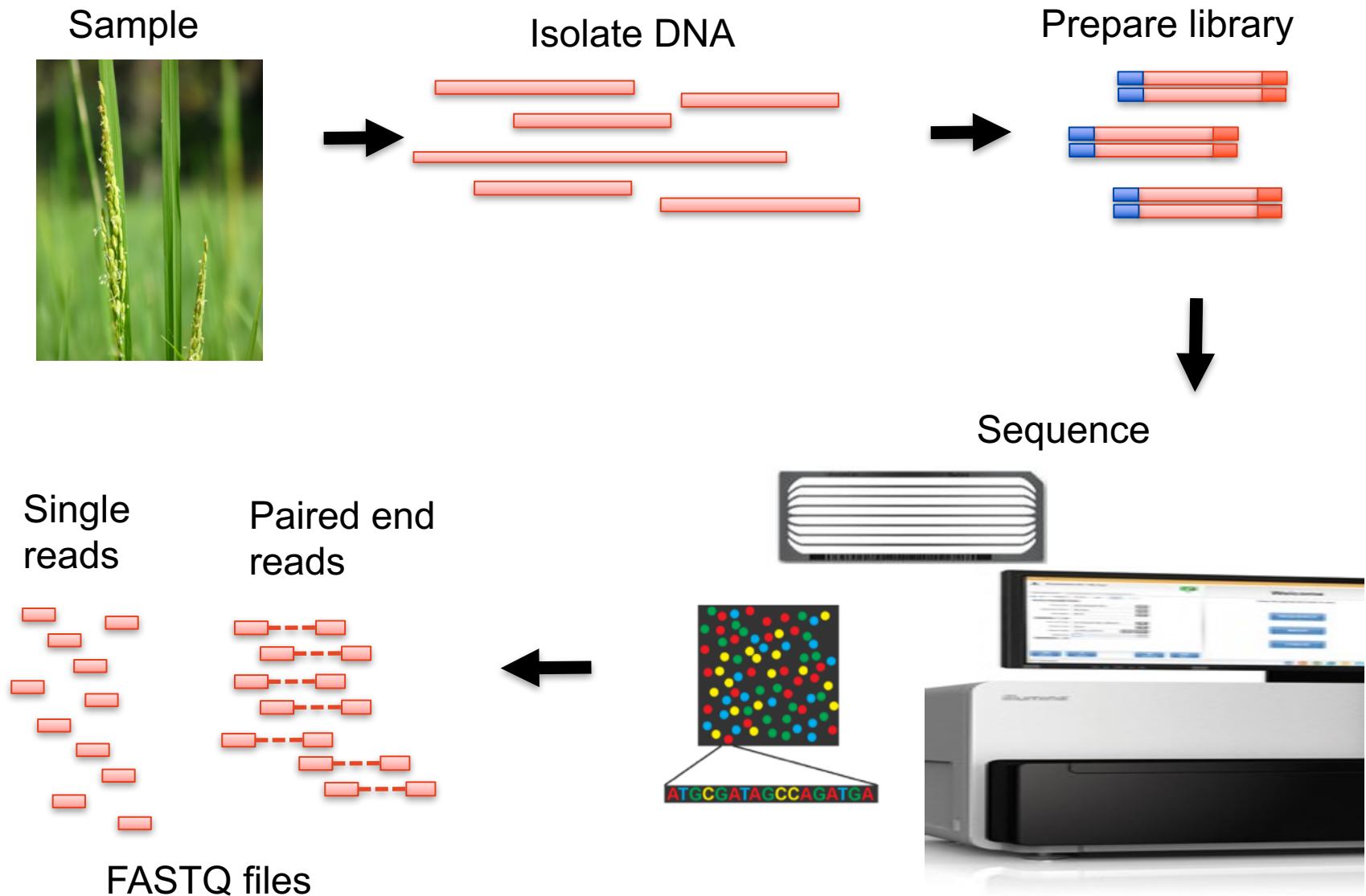
Sample:
[SAMN09303196](#) • SRS3417421 • [All experiments](#) • [All runs](#)
[Organism: Oryza sativa Indica Group](#)

Library:
Name: W_1
Instrument: Illumina HiSeq 2500
Strategy: WGS
Source: GENOMIC
Selection: RANDOM
Layout: PAIRED

Runs: 1 run, 28.4M spots, 5.1G bases, [3.3Gb](#)

Run	# of Spots	# of Bases	Size	Published
SRR7341627	28,352,087	5.1G	3.3Gb	2018-06-14

Part A: Illumina sequencing



Part A: FASTQ files

Inspect your fastq files:

```
cd /project/Training/<unikey>/DNA_workshop/Raw_fastq  
  
ls
```

You should see two fastq files and one txt file. Use head to view the top of a file, e.g:

```
head SRR7341627_1.fastq
```

Part A: FASTQ files

Inspect your fastq files:

```
[tche7417@login2 Raw_fastq]$ head SRR7341627_1.fastq
1 @SRR7341627.134.ACTTGAAT/1
2 GGTAGCTATCAGTGCTTATTTGTGGTGTTCATGCTGATTAAATGCTAAAATATGTCCAAGAATGAGCGTGGTCAAGTCGATGTGTGGT
3 +
4 ;;8;:::;9;:::::::::8;:99::;:9:::;::9::::::::9::::::::;::;<<<;<<8:<;:<9;;;=9;<=;;;::;
```

Line 1	@ followed by sequence identifier. Usually contains some sequencing and pair membership information (e.g. @HWUSI-EAS100R:6:73:941:1973#0/1)
Line 2	Raw sequence
Line 3	+ optionally followed by sequence identifier/description
Line 4	Quality values for line 2 encoded in ASCII (usually Phred+33)

How does SRR7341627_1.fastq compare to SRR7341627_2.fastq?

Part A: The reference sequence

The reference sequence...

- Contains DNA sequence that is representative of a species, organised by chromosome
- Are haploid (even if the species is not naturally)
- Are often created from several individuals, with the most commonly occurring alleles included
- Are often updated and periodically, different **versions** are released

You can download reference sequences and their annotations from [Ensembl](#) (or [EnsemblPlants](#)) and [UCSC](#)

Check the contents of the Reference directory:

```
cd /project/Training/<unikey>/DNA_workshop/Reference  
  
ls
```

Part A: FASTA files

Take a look at the top of the FASTA file

```
head Oryza indica.ASM465v1.dna.chr8.fasta
```

- The reference sequence is provided in FASTA format
- Today we are only going to work with chromosome 8 of the *Oryza indica* reference sequence (ASM465v1)
- `oryza_indica.vcf.gz` contains known variants (we will look at this later)
- For the purposes of this course, we will pretend the reference sequence represents a non-fragrant variety of rice

```
[tche7417@login4 Reference]$ head Oryza_indica.ASM465v1.dna.chr8.fasta  
>8 dna:chromosome chromosome:ASM465v1:8:1:30396518:1 REF  
CCTAACCTAACCTAACCTAACCTAACCTAAACCTAACCTAACCTAACCTAAC  
CCTAACCTAACCTAAACCCTAAACCCTAAACCCTAAACCCTAAACCCTAAACC  
CGAATAGCCGTGCTCGCGGTTATCGGGCATCGGGACTCACTATGGCATCTTGGATAGAAA  
CCTAACTTAAACATTACTAAATACTGTTGCATTTAAACATTATAGGCTTTCTTCCTCT  
ATATTATCTGTTTTCTGCAAATAAACCTAGCCCTATACAGTTATTACCATTTCATGCATT
```

Part B: Quality checking

Before we map our samples to the reference sequence, we will check the quality of the sequence using fastqc.pbs

Go to the Scripts directory:

```
cd /project/Training/<unikey>/DNA_workshop/Scripts  
ls
```

Open “fastqc.pbs” using your favourite text editor

```
nedit fastqc.pbs &
```

This script uses fastQC. FastQC creates a single quality report for a single fastq file at a time.

Part B: Job arrays

We can run fastQC for our two fastq files in parallel using PBS job arrays by adding:

#PBS -J 1-2

This can save us a lot of time (especially if you had hundreds of samples and fastq files!)

```
#!/bin/bash

# Run fastQC on reads
# Use job array to process files in parallel
# Supply fastq file names as list (fastq_list.txt)

# PBS directives
#PBS -P Training
#PBS -N fastQC
#PBS -l select=1:ncpus=1:mem=4GB
#PBS -l walltime=00:10:00
#PBS -M <email>
#PBS -m e
#PBS -q training
#PBS -J 1-2

# Load modules
module load fastqc/0.11.3

# Define variables
dir=/project/Training/<unikey>/DNA_workshop
out=${dir}/fastQC
list=${dir}/Raw_fastq/fastq_list.txt
fq=$(awk -v taskID=${PBS_ARRAY_INDEX} ' $1==taskID {print $2}' ${list})

# Perform task
# Make a directory using the variable ${out}
mkdir -p ${out}

# Use fastqc and generate quality reports
fastqc -o ${out} ${dir}/Raw_fastq/${fq}
```

Replace all instances of <__> (including the brackets) with values that are relevant to you.

Save the file (ctrl+s)

Part B: Job arrays 101

```
#PBS -J 1-2
```

This directive will cause “fastqc.pbs” to run twice at the same time, changing only one variable between the two jobs:



```
${PBS_ARRAY_INDEX}=1
```



```
${PBS_ARRAY_INDEX}=2
```

We can then use this variable to input other variables that are relevant to our particular job (e.g. each of the two fastq files)

Part B: Job arrays 101

The magic line:

Create a variable called taskID, save value of \$PBS_ARRAY_INDEX

```
fq=$(awk -v taskID=${PBS_ARRAY_INDEX} ' $1==taskID {print $2}' ${list})
```

If column 1 of the \${list} file = taskID, execute the next part

Print column 2 (saving it to the variable \${fq})

Print column 2 of \${list} file (saving it to the variable \${fq})

As defined earlier, `list=${dir}/Raw_fastq/fastq_list.txt` and looks like:

```
[tche7417@login2 Raw_fastq]$ cat fastq_list.txt
1  SRR7341627_1.fastq
2  SRR7341627_2.fastq
```

Part B: Job arrays 101

Save your newly formatted fastqc.pbs script (ctrl+s).

To keep things tidy, run your script from the “Logs” directory

```
cd /project/Training/<unikey>/DNA_workshop/Scripts/Logs
qsub ../fastqc.pbs
```

Check the status of your jobs using

```
qstat -tu <training_unikey>
```

Job ID	Username	Queue	Jobname	SessID	NDS	TSK	Memory	Time	S	Time
2410781[1].pbss	tche7417	scavenge	fastQC	196103	1	1	8gb	01:00	R	00:00
2410781[2].pbss	tche7417	scavenge	fastQC	196125	1	1	8gb	01:00	R	00:00

Part B: Interactive jobs 101

Fastqc creates quality reports in HTML. HTML files are opened by web browsers (e.g. Chrome, Firefox)

We will use the interactive queue to open firefox to view our report files. The interactive node is required to open graphical user interface (GUI) programs such as firefox.

Please type:

```
qsub -IXP Training -l select=1:ncpus=0:mem=4GB,walltime=1:00:00
```

Interactive session is ready to use once you see something like this:

```
qsub: waiting for job 2410848.pbserver to start
qsub: job 2410848.pbserver ready
```

```
[tche7417@hpc055 ~]$
```

Part B: Interactive jobs 101

Once an interactive job initiates, you are automatically taken to your `/home/<unikey>` directory (shortcut is `~` in command line)

```
[tche7417@hpc055 ~]$
```

Go to your newly created fastQC folder:

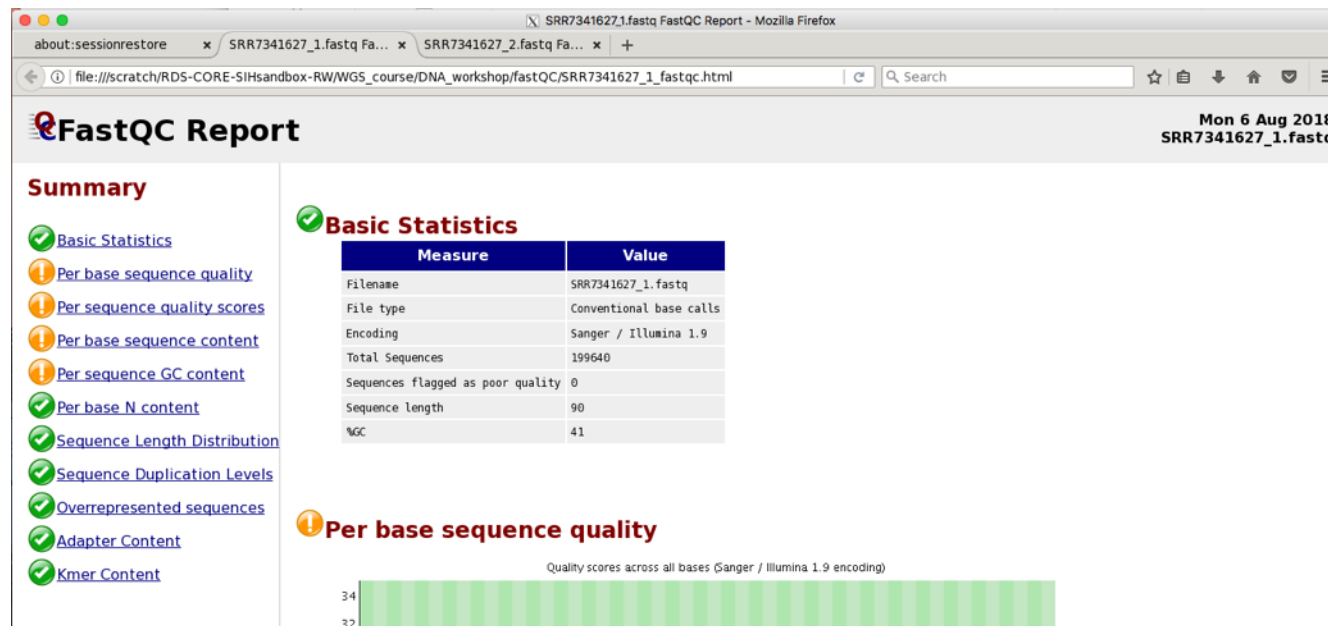
```
cd /project/Training/<unikey>/DNA_workshop/fastQC  
ls
```

The .zip files contain more comprehensive quality reports. Lets view the .html report files

```
firefox SRR7341627_1_fastqc.html &  
firefox SRR7341627_1_fastqc.html &
```

Part B: Interactive jobs 101

A webpage-like window will open with the quality report of the fastq file.



The authors of FASTQC have provided a description of each category.

Part B: FastQC

-  passed QC
-  failed QC
-  warning

Phred quality scores are logarithmically linked to error probabilities

Phred Quality Score	Probability of incorrect base call	Base call accuracy
10	1 in 10	90%
20	1 in 100	99%
30	1 in 1000	99.9%
40	1 in 10,000	99.99%
50	1 in 100,000	99.999%
60	1 in 1,000,000	99.9999%

Quality scores are Phred Scaled: $Q = -10 \log_{10} P$

- What are the lengths of our reads?
- Which part of the reads tend to have worse per base sequence quality, the start or the end?
- What is the approx. average base call accuracy?
- Where would I be able to detect evidence of contamination?
- Where would I be able to detect evidence of technical bias?

Exit the interactive session:

```
exit
```

Tip! MultiQC can summarise all fastQC reports into a single interactive HTML file.

Part C: Preparing the reference genome

Before we commence with mapping, we need to index the reference genome. First, edit the script:

```
cd /project/Training/<unikey>/DNA_workshop/Scripts  
nedit index_reference.pbs &
```

Indexing the reference genome is required for mapping to run faster (less time, less memory – think of an index in a book).

It only has to be performed once if you are mapping multiple samples to a single reference genome.

Part C: Indexing the reference genome

Edit the
index_reference.pbs
script.

Notice that the script
creates index files for
three different
programs
(indexing may be
unique to a program)

Save the script
(ctrl+s)

```
#!/bin/bash

# Indexes a reference genome
# Reads can be queried faster against an indexed reference genome
# This saves a lot of time and memory
# This script indexes the reference genome for BWA, SAMtools and GATK (Picard)

# PBS directives:
#PBS -P Training
#PBS -N IndexRef
#PBS -l select=1:ncpus=1:mem=4GB
#PBS -l walltime=00:05:00
#PBS -M <email>
#PBS -m e
#PBS -q training

# Load modules:
module load bwa/0.7.15
module load samtools/1.8
module load java/jdk1.8.0_111
module load picard/2.7.1

# Define variables
ref=Oryza_indica.ASM465v1.dna.chr8
cd /project/Training/<unikey>/DNA_workshop/Reference

# Perform task
# Index for BWA
bwa index -a bwtsw ${ref}.fasta

# Index for SAMtools
samtools faidx ${ref}.fasta

# Index for GATK (using Picard)
java -Xmx8g -jar /usr/local/picard/1.119/CreateSequenceDictionary.jar \
    R=${ref}.fasta \
    O=${ref}.dict
```

Part C: Indexing the reference genome

Change to the Logs directory and submit the job

```
cd /project/Training/<unikey>/DNA_workshop/Scripts/Logs  
qsub ../index_reference.pbs
```

Optional: check the status of your job (you'll only have >1 min!)

```
qstat -u <training_unikey>
```

Optional: check the files that have been created by the indexing

```
cd /project/Training/<unikey>/DNA_workshop/Reference  
ls
```

Part D: Reference mapping

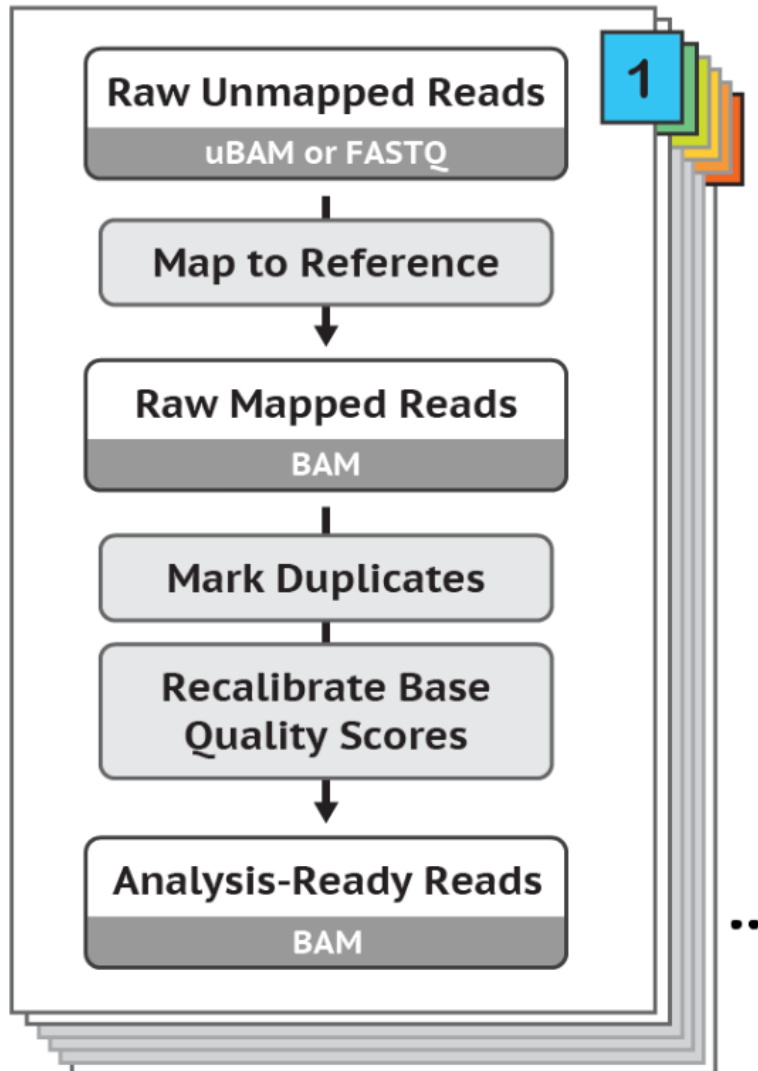
We will now map our raw paired end data (FASTQ files) to our indexed reference sequence using the align.pbs script

```
cd /project/Training/<unikey>/DNA_workshop/Scripts  
nedit align.pbs &
```

Edit and save the script.

You'll notice that this script is quite long – we will follow a workflow that includes some optional (but recommended) steps.

Part D: Reference mapping – this workflow

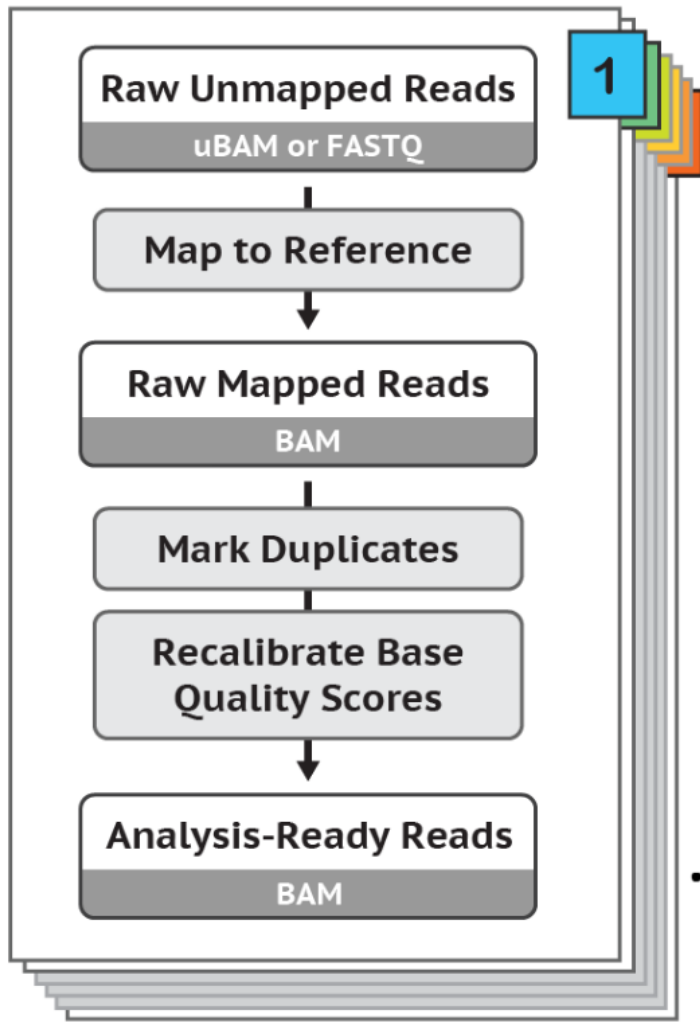


In this workshop, we will follow the Genome Analysis Toolkit ([GATK – Broad Institute](#)) best practices workflow.

This is just one workflow that you can use. It has been optimised for mapping short read (Illumina) data.

..

Part D: Reference mapping – this workflow



Software used

BWA-mem: mapping

SAMblaster: mark PCR duplicates

SAMtools: file management
including converting SAM > BAM,
indexing bam files

GATK: local realignment around
indels (improve alignment that is
prone to false +ve SNPs)

GATK: base quality score
recalibration (BQSR) using known
variants

Part D: Reference mapping – this workflow

Once you've looked through, edited and saved your align.pbs script, submit the job in the Logs directory

```
cd /project/Training/<unikey>/DNA_workshop/Scripts/Logs  
qsub ../align.pbs
```

You can check the status of this job by

```
qstat -u <training_unikey>
```

This script takes a few minutes to run, so please feel free to have a 10 minute break now.

Part D: Reference mapping – this workflow

Once your job is complete, your output will appear in a new directory called 'Alignments'

```
cd /project/Training/<unikey>/DNA_workshop/Alignments  
ls
```

The file "SRR7341627.final.bam" is your final alignment file (SRR7341627.final.bai is its corresponding index file).

The other .bam and .bai files are intermediary files and can be deleted once you have ensured that alignment has completed successfully

Part D: Check basic alignment stats

SAMtools can print some basic statistics about the alignment. First, load samtools, then run the flagstat tool:

```
module load samtools
```

```
samtools flagstat SRR7341627.final.bam
```

```
400181 + 0 in total (QC-passed reads + QC-failed reads)
901 + 0 secondary
0 + 0 supplementary
4040 + 0 duplicates
391817 + 0 mapped (97.91% : N/A)
399280 + 0 paired in sequencing
199640 + 0 read1
199640 + 0 read2
361750 + 0 properly paired (90.60% : N/A)
382560 + 0 with itself and mate mapped
8356 + 0 singletons (2.09% : N/A)
0 + 0 with mate mapped to a different chr
0 + 0 with mate mapped to a different chr (mapQ>=5)
```

Part D: Reference mapping – terminal viewer

A simple and fast way to visualise your alignments is by using SAMtools' terminal viewer (tview)

If you are not already in the Alignments directory:

```
cd /project/Training/<unikey>/DNA_workshop/Alignments
```

To view the alignment (the following is a single line):

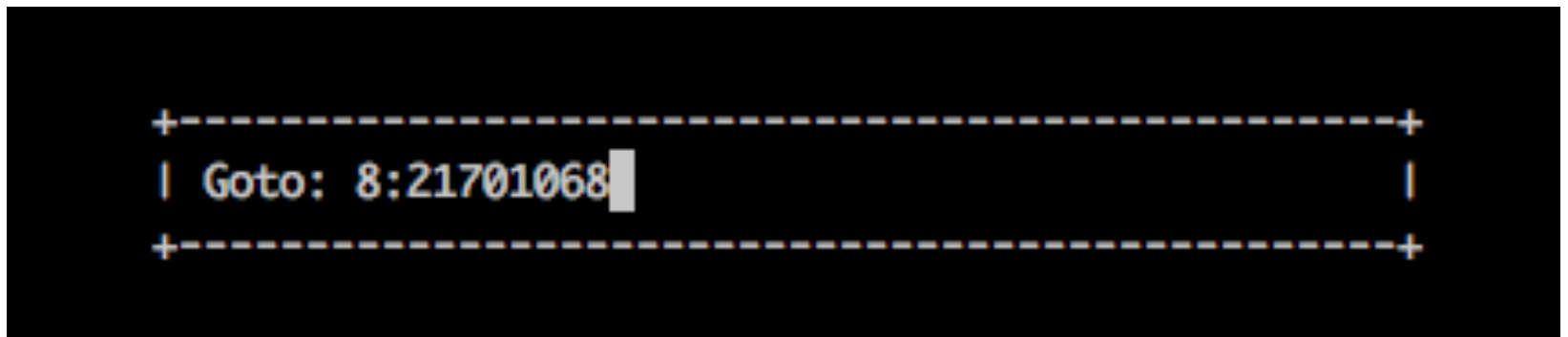
```
samtools tview SRR7341627.final.bam  
../Reference/Oryza_indica.ASM465v1.dna.chr8.fasta
```

Part D: Reference mapping – SAMtools tview

We can take a look at the *BADH2* to get an initial idea of the sort of coverage, alignment quality and what biological variants may be present.

BADH2 is located: 8:21701068-21706913

To go to this position, type g. A box with “Goto: “ should appear. Type in the start position of *BADH2* exactly as below:



Part D: Reference mapping – SAMtools tview

A help screen with instructions on how to navigate appear when you type '?'. Press enter to get out of this screen.

Use these to take a look at the alignment.

```
+-----+
|               Help               |
+-----+
| ?           This window          |
| Arrows      Small scroll movement |
| h,j,k,l     Small scroll movement |
| H,J,K,L     Large scroll movement |
| ctrl-H      Scroll 1k left        |
| ctrl-L      Scroll 1k right       |
| space       Scroll one screen     |
| backspace   Scroll back one screen |
| g           Go to specific location |
| m           Color for mapping qual |
| n           Color for nucleotide   |
| b           Color for base quality  |
| c           Color for cs color      |
| z           Color for cs qual      |
| .           Toggle on/off dot view |
| s           Toggle on/off ref skip  |
| r           Toggle on/off rd name   |
| N           Turn on nt view         |
| C           Turn on cs view         |
| i           Toggle on/off ins       |
| v           Inverse video           |
| q           Exit                    |
|                                     |
| -Underline:      Secondary or orphan-- |
| Blue:    0-9      Green: 10-19        |
| Yellow: 20-29     White: >=30        |
|                                     |
+-----+
```

Part D: Reference mapping – SAMtools tview

The first 'A' is at position 21702461

Secondary or orphan read (underline)

Locus with 8X coverage

An A > T variant

21702461 21702471 21702481 21702491 21702501 21702511 21702521 21702531
ATAATCGAGAGGAAATCTCAGCTGGCTAGACTAGAGACGCTTGATTGTGGGAAGCCTCTTGATGAAGCAGCA

Reference sequence

Consensus sequence

Base on reverse read
", " matching the
reference

Base on forward read
"." matching the
reference

Part D: Reference mapping – BAM/SAM files

To quit viewing the alignment, press 'q'.

All of the information about a read and its alignment is stored in the alignment file.

The standard file format for alignment files is BAM. The non-binary (human-readable) version of this file is SAM.

Part D: Reference mapping – BAM/SAM files

BAM/SAM files contain:

1. Optional headers (each line starting with @) that describe the file (e.g. reference sequences, programs used to generate the file)
2. Information about the alignment. One read is contained in one line. Each line contains 11 columns of information. The SAM format specification can be found [here](#).

Col	Field	Type	Regex/Range	Brief description
1	QNAME	String	[!-?A-~]{1,254}	Query template NAME
2	FLAG	Int	[0,2 ¹⁶ -1]	bitwise FLAG
3	RNAME	String	* [!-()+-<>-~] [!-~]*	Reference sequence NAME
4	POS	Int	[0,2 ³¹ -1]	1-based leftmost mapping POSition
5	MAPQ	Int	[0,2 ⁸ -1]	MAPping Quality
6	CIGAR	String	* ([0-9]+[MIDNSHPX=])+	CIGAR string
7	RNEXT	String	* = [!-()+-<>-~] [!-~]*	Ref. name of the mate/next read
8	PNEXT	Int	[0,2 ³¹ -1]	Position of the mate/next read
9	TLEN	Int	[-2 ³¹ +1,2 ³¹ -1]	observed Template LENgth
10	SEQ	String	* [A-Za-z=.]+	segment SEQUENCE
11	QUAL	String	[!-~]+	ASCII of Phred-scaled base QUALity+33

Part E: Variant calling

We will now call variants to determine whether our sample is from a fragrant or non-fragrant variety of rice.

Edit the 'variants.pbs' file by:

```
cd /project/Training/<unikey>/DNA_wokshop/Scripts  
nedit variants.pbs &
```

This script uses GATK Haplotype Caller to call SNPs and small indels. Again – this is just one variant calling workflow.

Additional steps may include variant quality score recalibration (GATK only), additional hard-filtering, multi-sample calling, etc etc...

Part E: Variant calling

--dbsnp annotates the final variant call file (VCF) with known variants (with their reference SNP id number).

GATK Haplotype Caller can also call genotypes in different ploidy (-ploidy)!

```
#!/bin/bash

# PBS directives
#PBS -P Training
#PBS -l select=1:ncpus=1:mem=4GB
#PBS -l walltime=00:10:00
#PBS -N Variants
#PBS -m e
#PBS -M <email>
#PBS -q training

# Load modules
module load gatk/4.0.2.1
module load java

# Define variables
io=/project/Training/<unikey>/DNA_workshop
ref=${io}/Reference/Oryza_indica_ASM465v1.dna.chr8.fasta
dbsnp=${io}/Reference/oryza_indica.vcf.gz
sample=SRR7341627
region=8:21694859-21711748

# Perform task
mkdir -p ${io}/Variants

java -Xmx4g -jar /usr/local/gatk/3.7.0/GenomeAnalysisTK.jar \
  -T HaplotypeCaller \
  -R ${ref} \
  --dbsnp ${dbsnp} \
  -I ${io}/Alignments/${sample}.final.bam \
  -L ${region} \
  -ploidy 2 \
  -o ${io}/Variants/${sample}.region.vcf
```

I

Part E: Variant calling

Submit the job from the Logs directory once you save the changes you have made to the 'variants.pbs' script (ctrl+s).

```
cd /project/Training/<unikey>/DNA_wokshop/Scripts/Logs  
qsub ../variants.pbs
```

This script may take a few minutes to run. In the meantime, you may wish to check the status of your job (do you remember the command to do this?)

Part E: Variant calling – VCF files

Once the job is complete, a new directory called ‘Variants’ will appear

```
cd /project/Training/<unikey>/DNA_workshop/Variants  
ls
```

There are two new files in this directory – the VCF file (.vcf) and its index file that is automatically created by GATK (.vcf.idx)

Let's take a look at the VCF file

Part E: Variant calling – VCF files

To print the contents of a file in the terminal:

```
cat SRR7341627.region.vcf
```

WARNING! I wouldn't normally recommend doing this as VCF files tend to be very large

There are three main sections of a VCF file:

##Metainformation

#Headers (of at least 8 of the mandatory columns)

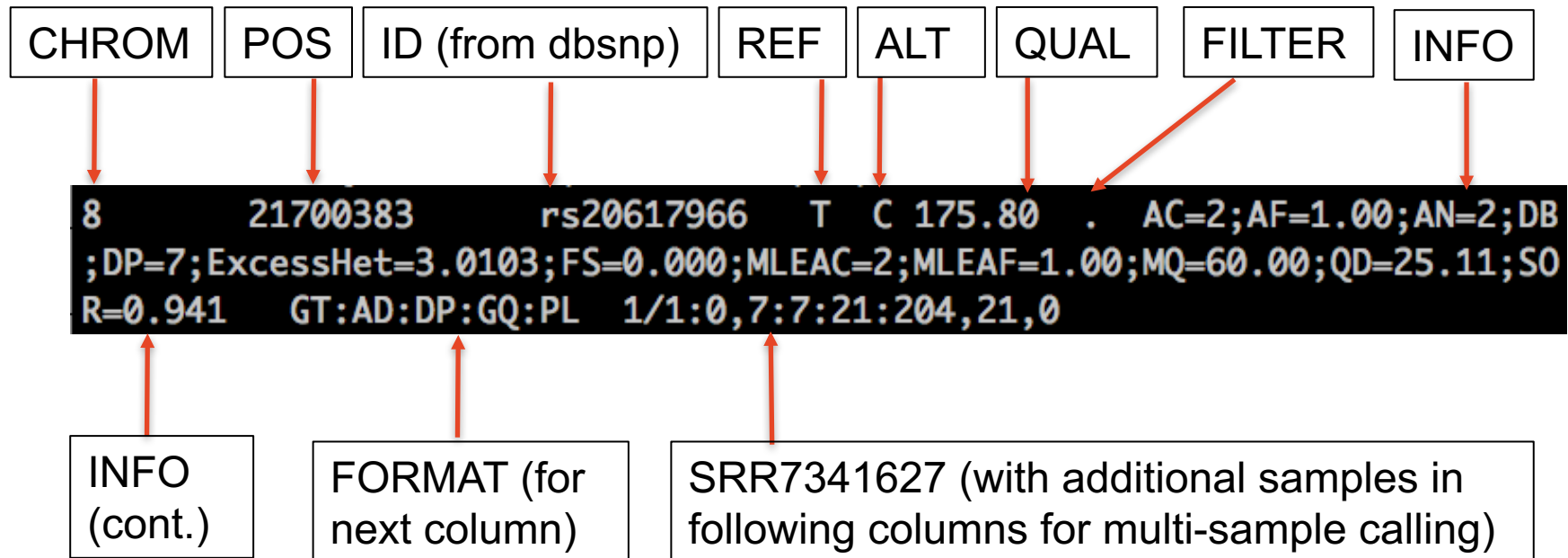
Data lines

Scroll down until you see some of the data lines.

Part E: Variant calling – VCF files

```
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT SRR7341627
8 21694891 . T C 46.77 . AC=1;AF=0.500;AN=2;BaseQRankSum=0.862;ClippingRankSum=0.000;DP=20;ExcessHet=3.0103;FS=2.831;MLEAC=1;MLEAF=0.500;MQ=57.03;MQRankSum=-4.257;QD=2
8 21694901 . T G 46.77 . AC=1;AF=0.500;AN=2;BaseQRankSum=0.985;ClippingRankSum=0.000;DP=20;ExcessHet=3.0103;FS=2.831;MLEAC=1;MLEAF=0.500;MQ=55.93;MQRankSum=-3.478;QD=2
8 21694927 . C T 51.77 . AC=1;AF=0.500;AN=2;BaseQRankSum=-1.578;ClippingRankSum=0.000;DP=21;ExcessHet=3.0103;FS=9.084;MLEAC=1;MLEAF=0.500;MQ=52.55;MQRankSum=-3.539;QD=2
8 21695057 rs20617138 A C 379.77 . AC=2;AF=1.00;AN=2;DB;DP=14;ExcessHet=3.0103;FS=0.000;MLEAC=2;MLEAF=1.00;MQ=48.70;QD=27.13;SOR=1.765 GT:AD:DP:GQ:PL 1/1:0,14:14:
8 21695080 . C T 324.77 . AC=2;AF=1.00;AN=2;DP=12;ExcessHet=3.0103;FS=0.000;MLEAC=2;MLEAF=1.00;MQ=48.31;QD=27.06;SOR=1.981 GT:AD:DP:GQ:PL 1/1:0,12:12:36:353,36,0
```

One line in the ‘data line’ section corresponds to a variant at a single locus. It is a very long line and may wrap around to the next line or two



Part E: Variant calling – VCF files

```
8      21700383      rs20617966      T      C      175.80      .      AC=2;AF=1.00;AN=2;DB  
;DP=7;ExcessHet=3.0103;FS=0.000;MLEAC=2;MLEAF=1.00;MQ=60.00;QD=25.11;SOR  
R=0.941      GT:AD:DP:GQ:PL      1/1:0,7:7:21:204,21,0
```

We can search header lines to get a description of the acronyms used. For example:

```
grep ID=GT SRR7341627.region.vcf
```

Why do we need to include 'ID='?

```
[tche7417@login4 Variants]$ grep ID=GT SRR7341627.region.vcf  
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
```

You can also read more about VCF files [here](#).

Part F: Variant annotation - VEP

We will use Ensembl's Variant Effect Predictor (VEP) to annotate variants in your web browser.

Go to: http://plants.ensembl.org/Oryza_indica/Info/Index

Scroll down and click: Variant Effect Predictor



Fill in the relevant information. We will input the variant data from our VCF file.

Part F: Variant annotation - VEP

Variant Effect Predictor ?

Species:

 Oryza sativa Indica 

Assembly: ASM465v1

Name for this job (optional):

Rice

Either paste data:

```
Q:PL 1/1:0,11:11:33:355,33,0
QRankSum=-3.138;QD=2.15;ReadPosRankSum=-1.
QRankSum=-2.362;QD=2.62;ReadPosRankSum=-2.
1:0,15:15:45:452,45,0
1:0,16:16:48:460,48,0
1:0,16:16:48:477,48,0
```

Copy data lines
from VCF file in
the terminal
and paste it
here

Examples: [Ensembl default](#), [VCF](#), [Variant identifiers](#), [HGVS notations](#)

Or upload file:

Choose file No file chosen

Or provide file URL:

Identifiers and frequency data 

Additional identifiers for genes, transcripts and variants; frequency data

Extra options 

e.g. SIFT, PolyPhen and regulatory data

Filtering options 

Pre-filter results by frequency or consequence type

Click run

Run > [Clear](#)

Part F: Variant annotation - VEP

Under job details, you obtain a command line equivalent for the job performed

Job details

Job summary

VEP analysis of Rice in Oryza_indica



Species

 Oryza sativa Indica

Assembly

ASM465v1

Options summary

1000 Genomes global minor allele frequency:	Enabled
APPRIS:	Enabled
Find co-located known variants:	Enabled
Gene symbol:	Enabled
PubMed IDs for citations of co-located variants:	Enabled
Restrict results:	Disabled
SIFT:	Prediction and score
Transcript biotype:	Enabled
Transcript support level:	Enabled
Upstream/Downstream distance (bp):	5000

Command line equivalent

```
--check_existing --distance 5000 --sift b --species oryza_indica --symbol --tsl --cache --input_file [input_data]
```

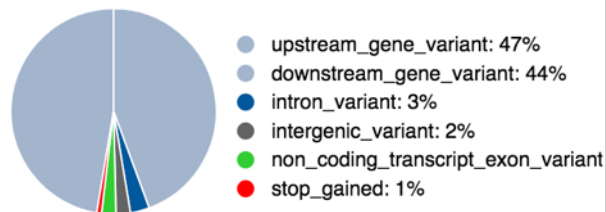
Part F: Variant annotation - VEP

Let's take a look at the results under Summary statistics.

Summary statistics

Category	Count
Variants processed	42
Variants filtered out	0
Novel / existing variants	34 (81.0) / 8 (19.0)
Overlapped genes	6
Overlapped transcripts	6
Overlapped regulatory features	-

Consequences (all)



Coding consequences



The table below describes each variant in more detail.

Click “All” to display all variant annotations

Part F: Variant annotation - VEP

.	8:21702512-21702512	T	 stop_gained	HIGH	BADH2
.	8:21702512-21702512	T	 intron_variant	MODIFIED	BADH2

- From this, can you determine whether our rice is fragrant or non-fragrant?

Please help us help you!

Please fill in:

- the attendance sheet
- Feedback survey



Research Computing Services

Provides research computing expertise, training, and support

- Data analyses and support (bioinformatics, modelling and simulation, visualisation)
- Training and workshops
 - High Performance Computing (HPC)
 - Programming (R, Python, Matlab, Scripting, GPU)
 - Code management (Git)
 - Bioinformatics (RNA-Seq, Genomics)
- Research Computing Support
 - Artemis HPC
 - Argus Virtual Research Desktop
 - Bioinformatics software support (CLC Genomics Workbench, Ingenuity Pathways Analysis)
- Events and Competitions
 - HPC Publication Incentive – High quality papers that acknowledge SIH and/or HPC/VRD
 - Artemis HPC Symposium

Data Science Expertise

Provides data science (e.g. machine learning, deep learning, AI, NLP) expertise, training, and support

Research Data Management and Digital Tools Support

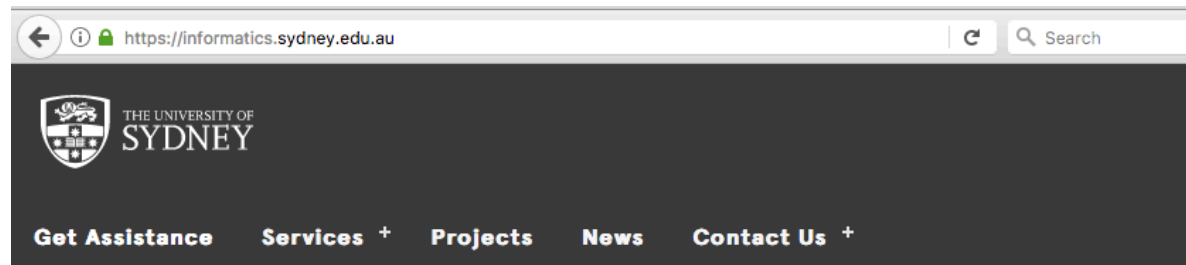
Provide expertise, training, and support on management of research data and use of digital tools.

- Digital research platforms supported
 - eNotebook - collaborative electronic notebook
 - REDCap - surveys and databases
 - GitHub - software repository management
 - Research Data Store
 - Dropbox
 - CloudStor
 - Office365/OneDrive

Sydney Informatics Hub

W: <https://informatics.sydney.edu.au>

E: sih.info@sydney.edu.au



Sydney Informatics Hub

The Sydney Informatics Hub (SIH) is a core research facility of the University of Sydney, providing **services** surrounding data and computation within the University. It delivers policies, systems, advice, engineering and training to our researchers and their external collaborators.

Researchers can access SIH's services primarily through attending **training workshops** or seeking assistance below.